Universidade Federal do Ceará - Campus Crateús

Curso de Sistemas de Informação

Disciplina: Algoritmos e Complexidade Computacional

Professor: Rafael Martins Barros

Trabalho Prático 01

Algoritmos Union-Find

Autores:

Carlos Eduardo de Almeida Coutinho
Stephany Rodrigues Gonçalves

Crateús - CE

Conteúdo

1 Introdução **2**

2 Método Experimental **2**

3 Resultados **3**

3.1 Casos de Teste Padrão 4

3.1.1 Tempo de Execução 4

3.2 Casos de Teste Adversáriais 5

3.2.1 Custo Amortizado 5

3.2.2 Tempo de Execução 9

4 Discussão **11**

4.1 Análise do Quick-Find 11

4.2 Análise do Quick-Union 12

4.3 Análise do Quick-Union Ponderado 12

5 Conclusão **13**

6 Uso de ferramentas de IA **13**

1 Introdução

O problema de conectividade dinâmica consiste em determinar, de forma eficiente, se dois elementos de um conjunto estão conectados entre si, considerando que novas conexões podem ser inseridas ao longo do tempo. Formalmente, dado um conjunto de N elementos e uma sequência de pares de entrada (p, q), o objetivo é identificar, para cada par, se p e q já pertencem ao mesmo componente conectado e, caso contrário, uni-los.

Para solucionar esse problema, foram implementadas e analisadas empiricamente três abordagens da estrutura de dados Union-Find: o Quick-Find, o Quick-Union e o Quick-Union Ponderado. O Quick-Find prioriza a operação de busca, mantendo um vetor de identificadores que permite verificar a conectividade em tempo constante O(1), ao custo de uniões lentas O(N). O Quick-Union inverte esse compromisso, representando os componentes como uma floresta de árvores e permitindo uniões mais ágeis contudo, sujeitas à piora em pior caso. O Quick-Union Ponderado aprimora essa última abordagem ao incorporar uma heurística de balanceamento baseada no tamanho dos componentes, limitando a altura das árvores a $logN$ e garantindo complexidade amortizada eficiente para ambas as operações.

O objetivo deste trabalho é comparar o desempenho dessas três implementações por meio de experimentos com instâncias de tamanhos variados, tanto casos padrão quanto entradas adversariais deliberadamente construídas para explorar os piores casos de cada algoritmo, analisando métricas de tempo de execução e custo amortizado por acesso ao vetor principal.

2 Método Experimental

Os experimentos foram executados em um ambiente Linux, utilizando o CachyOS como distribuição, sendo ela baseada no Arch Linux. O notebook usado possui um processador Ryzen 5 5500U com 18GB de RAM e todos os testes foram feitos conectados a energia para não haver perca de performace. A linguagem utilizada para a implementação foi o Python, escolhida devido a familiaridade dos alunos para com a linguagem, bem como por conta da facilidade de geração dos gráficos para a análise de desempenho utilizando a biblioteca `matplotlib`.

Os tamanhos das instâncias testadas foram:

- Instâncias Padrões:
  - `tinyUF.txt` = 10 elementos
  - `mediumUF.txt` = 625 elementos
  - `largeUF.txt` = 1.000.000 elementos
- Instâncias adversariais:
  - `adversarial_10000.txt` = 10^4
  - `adversarial_50000.txt` = 5×10^4
  - `adversarial_100000.txt` = 10^5
  - `adversarial_500000.txt` = 5×10^5

As métricas de tempo de execução `id[]` foram coletadas executando cada algoritmo ao menos 10 vezes por experimento conforme requisitado no documento de instruções. O custo foi determinado ao rodar apenas uma vez, pois por ser um processo determinístico, rodar uma ou dez vezes resultaria no mesmo resultado para cada iteração.

Criamos 3 arquivos principais: `main.py` onde é feita a análise de tempo das instâncias padrões fornecidas pelo professor(TinyUF, mediumUF e largeUF) as passando 10 vezes para cada algoritmo; `plotar_custos.py` onde são executados os testes das instâncias adversáriais 1 vez e gera os gráficos ao final da execução exibindo o número de acessos ao vetor `id[]` conforme o número de conexões processadas aumenta; `plotar_tempo.py` executa 10 vezes o algoritmo nas instâcias adversáriais, gerando um gráfico ao final da execução exibindo o tempo de execução pela quantidade de elementos das instâncias.

Foi utilizado o módulo `time.perf_counter()` para obtenção mais precisa e detalhada na geração dos casos adversarios (os teste de 10.000 até 500.000). Ela foi desenvolvida especificamente para benchmark sendo a ferramenta recomendada para teste de desempenho em vez do time.time().[2]

3 Resultados

Abaixo estão apresentados os resultados obtidos a partir das métricas definidas, divididos entre o tempo de execução e a análise do custo amortizado através do acesso ao

vetor. Bem como em tipo: as instâncias padrão e as instâncias adversariais. Só foi feito os teste de tempo para os casos de teste padrão, pois, por ser casos pequenos não viu-se muita utilidade em comparar eles, tendo em vista que o caso maior (1.000.000) só foi executado no Quick-Union Ponderado.

3.1 Casos de Teste Padrão

3.1.1 Tempo de Execução

Tabela 1: Tempo médio de execução para cada algoritmo.

Instância (N)	**Quick-Find**	**Quick-Union**	**Q.U. Ponderado**
tinyUF	0.00001s	0.00001s	0.00001s
mediumUF	0.01887s	0.0094s	0.00040s
largeUF	iniviável	inviável	2.64055s

A tabela apresenta o tempo médio de execução para cada algorimo, evidenciando a ineficiênica do Quick-Find e Quick-Union para intâncias relativamente grandes. Para instâncias intermédiarias (mediumUF), o Quick-Union demonstrou uma execelente melhoria em comparação ao Quick-Find. Apesar disso, ele ainda continua ineficiênte para a instância grande, revelando a maior fraqueza do Quick-Find e Quick-Union, seu tempo $O(n^2)$. O Quick-Union Ponderado se mostrou excepcional tanto para instâncias pequenas quanto para grandes, resolvendo um problema com 1.000.000 de elementos em poucos segundos.

Tabela 2: Desvio Padrão para cada algoritmo.

Instância (N)	**Quick-Find**	**Quick-Union**	**Q.U. Ponderado**
tinyUF	0.00000s	0.00000s	0.00000s
mediumUF	0.0009s	0.00015s	0.00007s
largeUF	iniviável	inviável	0.20315s

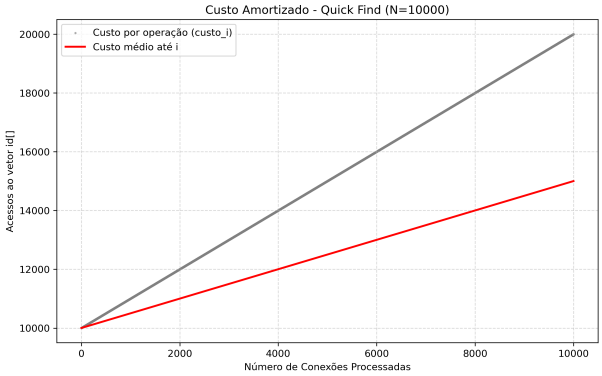
A tabela apresenta o desvio padrão dos tempos de execução, evidenciando a estabilidade dos algoritmos. Para instâncias intermediárias (mediumUF), o Quick-Union Ponderado demonstrou ser significativamente mais previsível e imune a flutuações do hardware em comparação ao Quick-Find. Na instância massiva (largeUF), a complexidade quadrática tornou a coleta de dados inviável para as implementações do Quick-Find e Quick-Union, destacando a resiliência do algoritmo ponderado, que manteve uma variação controlada de apenas $0.20315s$.

3.2 Casos de Teste Adversáriais

3.2.1 Custo Amortizado

Para avaliarmos o custo de processamento dos algoritmos, foram coletadas as métricas de acesso de leitura e escrita ao vetor principal durante a execução dos casos de teste adversarias. Os gráficos a seguir irão ilustrar o comportamento do custo por operação individual(pontos em cinza) e a evolução do custo médio(linha em vermelho) à medida que novas conexões são processadas.

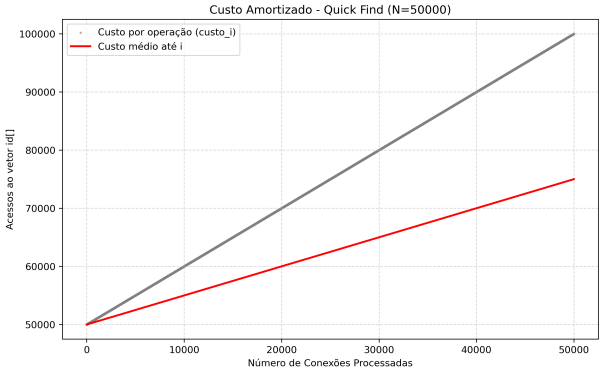
Figura 1: Gráfico do Custo médio - Quick-Find - N = 10.000



Fonte: Elaborado pelos autores (2026)

O gráfico demonstra o elevado número de acessos à memória (linha cinza) exigidos a cada operação de união, resultando em um custo médio (linha vermelha) linearmente crescente ao longo das conexões processadas.

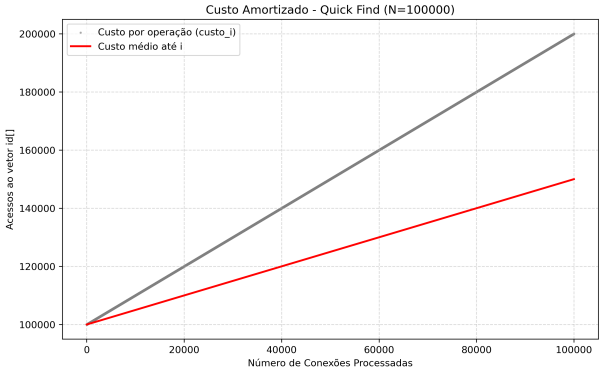
Figura 2: Gráfico do Custo médio - Quick-Find - N = 50.000



Fonte: Elaborado pelos autores (2026)

Aumentando para 50.000 conexões, o comportamento do custo médio se mantém crescente, confirmando a ineficiência da varredura completa do vetor `id[]` para instâncias maiores.

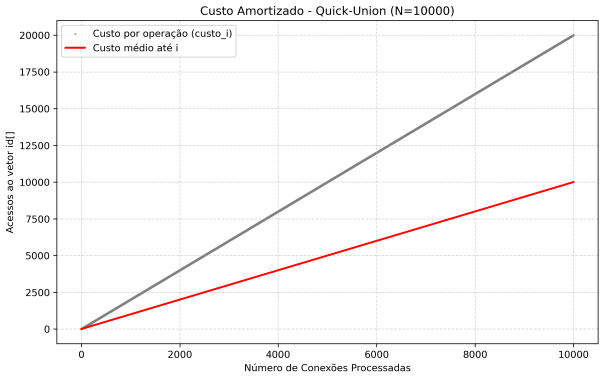
Figura 3: Gráfico do Custo médio - Quick-Find - N = 100.000



Fonte: Elaborado pelos autores (2026)

Com 100.000 elementos, o custo por operação atinge a marca de centenas de milhares de acessos a cada nova conexão processada. A manutenção desse custo elevado e linearmente crescente evidencia o limite prático da estrutura de dados para o processamento de entradas deste porte.

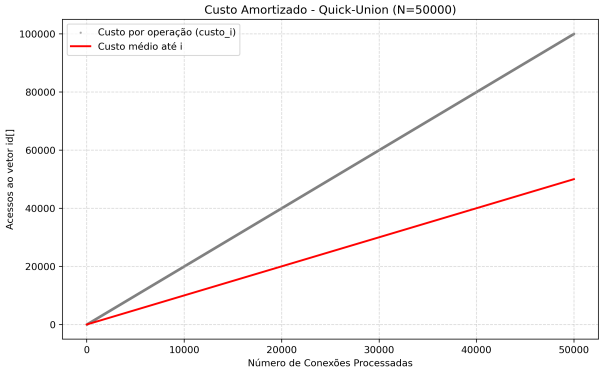
Figura 4: Gráfico do Custo médio - Quick-Union - N = 10.000



Fonte: Elaborado pelos autores (2026)

O gráfico ilustra o aumento progressivo nos acessos ao vetor, sintoma característico da formação de árvores desbalanceadas na estrutura durante o cenário adversarial testado.

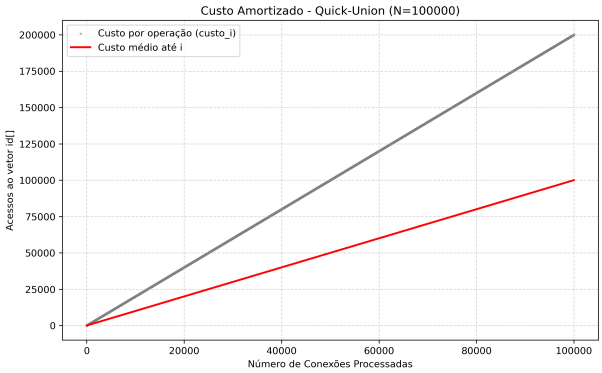
Figura 5: Gráfico do Custo médio - Quick-Union - N = 50.000



Fonte: Elaborado pelos autores (2026)

Para 50.000 elementos, nota-se a degradação contínua do custo médio, reflexo direto do aumento da profundidade das árvores que precisam ser percorridas pelas operações de busca (find).

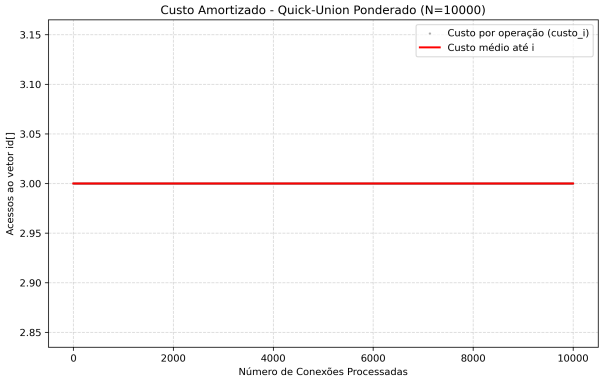
Figura 6: Gráfico do Custo médio - Quick-Union - N = 100.000



Fonte: Elaborado pelos autores (2026)

O escalonamento da entrada demonstra que, no pior caso, o Quick-Union também sofre de ineficiência de acesso, forçando o algoritmo a percorrer longos caminhos devido à ausência de balanceamento.

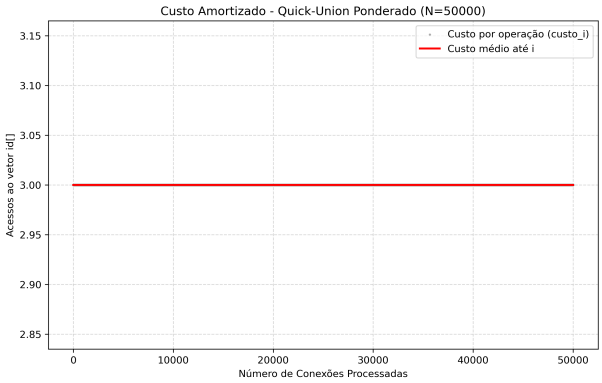
Figura 7: Gráfico do Custo médio - Quick-Union Ponderado - N = 10.000



Fonte: Elaborado pelos autores (2026)

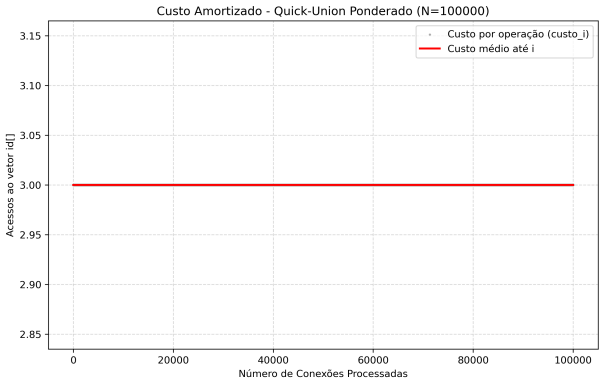
O custo por operação (linha cinza) e o custo médio (linha vermelha) mantêm-se estabilizados em exatos 3 acessos, evidenciando o balanceamento perfeito proporcionado pelo vetor de tamanhos.

Figura 8: Gráfico do Custo médio - Quick-Union Ponderado - N = 50.000



Fonte: Elaborado pelos autores (2026)

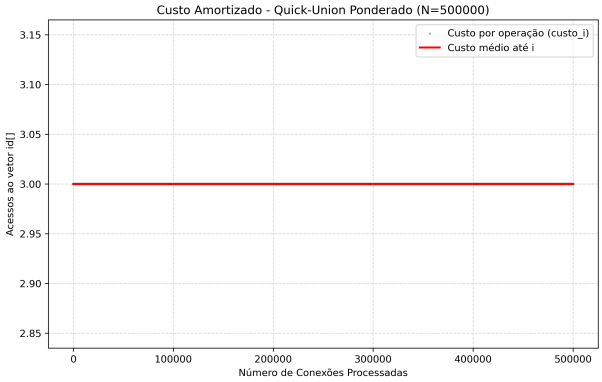
Figura 9: Gráfico do Custo médio - Quick-Union Ponderado - N = 100.000



Fonte: Elaborado pelos autores (2026)

Mesmo com o aumento expressivo da instância, a otimização da árvore garante a manutenção do custo amortizado estritamente constante, evitando o efeito cascata visto nos algoritmos anteriores.

Figura 10: Gráfico do Custo médio - Quick-Union Ponderado - N = 500.000



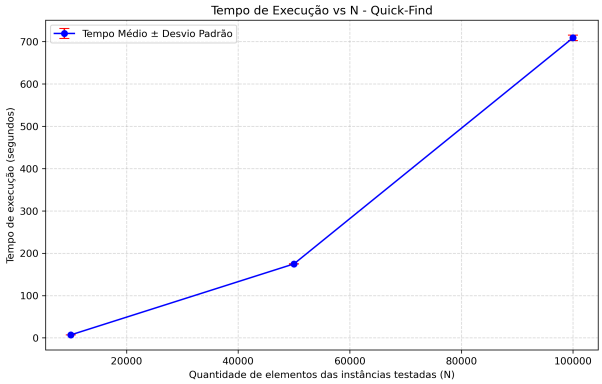
Fonte: Elaborado pelos autores (2026)

O processamento de meio milhão de conexões adversariais operando com número estático de acessos à memória prova de forma empírica a eficácia da abordagem ponderada em limitar a altura máxima da estrutura.

3.2.2 Tempo de Execução

Para avaliarmos o desempenho temporal dos algoritmos, foram coletadas as métricas de tempo de execução, em segundos, durante o processamento de instâncias de tamanhos crescentes nos casos de teste adversariais. Os gráficos a seguir irão ilustrar o escalonamento do tempo médio de processamento (linha principal) e a sua respectiva variação estatística através do desvio padrão (barras de erro) à medida que a quantidade de elementos da entrada (N) aumenta.

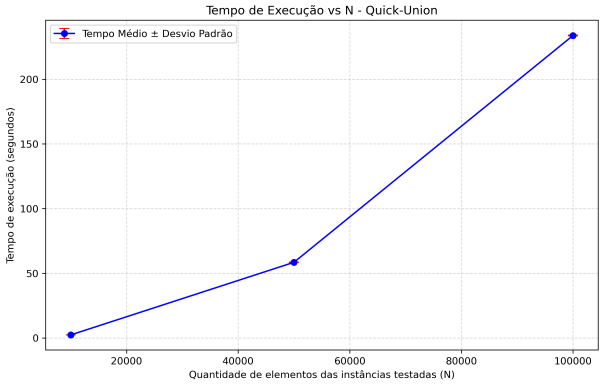
Figura 11: Gŕafico de tempo médio - Quick-Find



Fonte: Elaborado pelos autores (2026)

Observa-se um crescimento quadrático acentuado no tempo de execução médio. Instâncias de 100.000 elementos ultrapassaram os 700 segundos por execução, justificando a classificação de timeout (tempo limite excedido) para testes maiores.

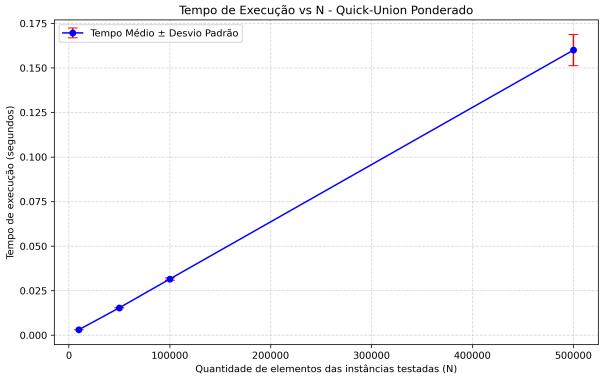
Figura 12: Gŕafico de tempo médio - Quick-Union



Fonte: Elaborado pelos autores (2026)

A curva de tempo exibe um crescimento acelerado que valida a degradação observada nos acessos à memória. A transformação da árvore em uma estrutura semelhante a uma lista encadeada resulta em severa penalidade temporal. O teste para 100.000 não foi executado pois será análogo ao do Quick-Find, levando mais de duas horas e meia para rodar os testes.

Figura 13: Gŕafico de tempo médio - Quick-Union Ponderado



Fonte: Elaborado pelos autores (2026)

Em contraste com as implementações prévias, o tempo de execução apresenta uma progressão suave e controlada, processando volumes massivos de dados de forma altamente performática.

4 Discussão

A análise empírica dos algoritmos revelou comportamentos de desempenho que corroboram com o que foi explicitado em sala, a complexidade quadrática do Quick-Find e Quick-Union e a eficiente execução do Quick-Union Ponderado($O(logn)$). O uso de entradas adversariais foi fundamental para avaliar o pior caso de execução de cada implementação.

4.1 Análise do Quick-Find

O algoritmo Quick-Find apresentou um desempenho severamente degradado em entradas grandes. Conforme observado na figura 11, o tempo de execução escalou de forma quadrática: instâncias de N=10.000 foram resolvidas em poucos segundos, enquanto a instância de N=100.000 exigiu em média mais de 700 segundos por execução, inviabilizando testes com N=500.000.

Essa limitação prática ocorre devido à natureza de sua operação union. Para garantir que a função find opere em tempo constante $O(1)$, o Quick-Find é forçado a varrer todo o vetor `id[]` a cada nova união para atualizar os identificadores de componente. A figura 3 confirma essa ineficiência, mostrando que o custo de acesso à memória para uma única operação atinge a marca de centenas de milhares de leituras e escritas. Assim, para N

elementos, o algoritmo executa operações na ordem de $O(N^2)$, tornando-o obsoleto para volumes massivos de dados.

4.2 Análise do Quick-Union

Diferente da abordagem anterior, o Quick-Union modifica a estrutura de dados interpretando o vetor `id[]` não como um identificador direto de grupo, mas como uma floresta de árvores, onde cada posição armazena o identificador de seu nó pai. Essa alteração permite que a operação `union` seja executada apenas alterando a raiz de um componente.

Contudo, a análise das figuras 6 e 12 revela que essa modificação estrutural traz uma vulnerabilidade crítica. Submetido ao cenário adversarial, o algoritmo degenerou as árvores em uma estrutura equivalente a uma lista encadeada (onde cada nó aponta para o anterior, aumentando a profundidade em 1 a cada passo). Isso transferiu o gargalo de desempenho para o método `find`. Embora mais rápido que o Quick-Find por evitar reescritas massivas no vetor (finalizando N=100.000 em pouco mais de 4 minutos, em comparação aos 13 do Quick-Find), o Quick-Union também esbarra na complexidade de pior caso $O(N^2)$, exigindo que o algoritmo percorra caminhos cada vez mais longos para encontrar a raiz.

4.3 Análise do Quick-Union Ponderado

A introdução do vetor auxiliar `size[]` no algoritmo Quick-Union Ponderado mitigou drasticamente o problema de desbalanceamento, resultando é uma melhora de desempenho que beira a perfeição empírica para o caso de teste proposto. Ao assegurar que a árvore de menor tamanho seja sempre anexada à raiz da árvore maior, a altura máxima das árvores na floresta fica estritamente limitada a $\log_2 N$.

No caso adversarial em questão (conexões $0 - i$), a implementação ponderada transformou a estrutura em uma "estrela", onde todos os nós folha se conectam diretamente à raiz 0. Como evidenciado na figura 10, o custo operacional e o custo médio por união mantiveram-se cravados em exatos 3 acessos ao vetor principal, independente do momento da execução. Isso permitiu o processamento fluido de 500.000 conexões com complexidade amortizada de $O(1)$.

5 Conclusão

Os resultados experimentais documentados confirmam rigorosamente as previsões teóricas. A transição de um vetor de identidades (Quick-Find) para uma floresta de apontadores (Quick-Union) introduziu uma vantagem estrutural que só foi plenamente aproveitada quando associada a uma heurística de balanceamento (Quick-Union Ponderado). Fica evidente que, para cenários reais e sistemas de larga escala, o **Quick-Union Ponderado** é a única escolha viável entre as três abordagens, resolvendo com eficiência o problema de conectividade dinâmica.

6 Uso de ferramentas de IA

Conforme os requisitos do trabalho e a Portaria N° 39/PRPPG/UFC, declaramos abaixo o uso de ferramentas de Inteligência Artificial Generativa como apoio ao desenvolvimento deste relatório:

- **Quais ferramentas foram utilizadas:** Gemini
- **Para quais finalidades:** Geração do template em LaTeX, validação na previsão de tempo de execução
- **Prompts utilizados:**
  - "Gere um template em Latex para o relatório desse trabalho, de acordo com os requisitos de entrega do professor: (Foi passado o texto ao relatório contido no docuemnto de instruções do trabalho disponibilizado no SIGAA)"
  - "Dado que a execução do Quick-Find para uma instância de 10 000 elementos levou 8 segundos, para 50 000 levou 3 minutos e para 100 000 levou 13 minutos. Qual seria mais ou menos a estimativa para 500 000? Pelos meus cálculos como irá aumentar 5 vezes e o algoritmo é quadrático multiplicando 13 por 25 daria algo em torno de 325 minutos para executar, está correto?"

Referências

[1] SEDGEWICK, R.; WAYNE, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.

[2] STACK OVERFLOW. *time.time() or time.perf_counter() which is faster?*. Disponível em: https://stackoverflow.com/questions/66036844/time-time-or-time-perf-counter-which-is-faster. Acesso em: 18 abr. 2026.

[3] GITHUB. Repositório Gist de leonid-ed. *Snippet/Código fonte*. Disponível em: https://gist.github.com/leonid-ed/d199421416f91aa8486ba56d1e1bfe78. Acesso em: 18 abr. 2026.

[4] STACK OVERFLOW. *Testing Weighted Quick Union algorithm*. Disponível em: https://stackoverflow.com/questions/31186801/testing-weighted-quick-union-algorithm. Acesso em: 18 abr. 2026.